COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S1
DATE PERFORMED:	Aug 4, 2026	**DATE SUBMITTED:**	Aug 11, 2026
NAME:	Jave Joaquin Obias	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 5 **[Queues]**

1. PROCEDURE OUTPUT

SOURCE CODE

```cpp
#include <iostream>
#include <queue>

//display function
void display(std::queue<char> r);

int main() {

   std::queue<char> q;
   q.push('c');
   q.push('p');
   q.push('e');
   q.push('0');
   q.push('1');
   q.push('0');

   //display
   std::cout << "Display: ";
   display(q);

   //empty
   std::cout << "Queue is empty? (y/n): " << q.empty();
   std::cout << '\n';

   //size
   std::cout << "Queue size: " << q.size();
   std::cout << '\n';

   //front
   std::cout << "Front queue: " << q.front();
   std::cout << '\n';

   //back
   std::cout << "Back queue: " << q.back();
   std::cout << '\n';
```

```
    //pop
    std::cout << "Popping the front of queue: " << q.front();
    q.pop();
    std::cout << '\n';

    //display
    std::cout << "Display after popping: ";
    display(q);

    //push another element
    std::cout << "Pushing another element: 1";
    q.push('1');
    std::cout << '\n';

    //display
    std::cout << "Display after pushing: ";
    display(q);

    //delete the queue
    while (!q.empty()) {
        q.pop();
    }

    //empty
    std::cout << "Queue is empty? (y/n): " << q.empty();
    std::cout << '\n';

    return 0;
}

void display(std::queue<char> r) {
    //create a copy of the queue
    std::queue<char> c = r;

    while (!c.empty()) {
        std::cout << c.front() << " ";
        c.pop();
    }
    std::cout << '\n';
}
```

OUTPUT

```
Display: c p e 0 1 0
Queue is empty? (y/n): 0
Queue size: 6
Front queue: c
Back queue: 0
Popping the front of queue: c
Display after popping: p e 0 1 0
Pushing another element: 1
Display after pushing: p e 0 1 0 1
Queue is empty? (y/n): 1
```

ANALYSIS

In this program, it uses a queue to store the characters 'c', 'p', 'e', '0', '1', and '0' using the push() function, the front() and back() functions are used to check the first and last elements, while pop() removes the first element because a queue follows the first in, first out principle. The program also uses empty() and size() to check the queue, and it adds another '1' after removing the first element.

Table 5.1 ILO A: Create queue using C++ STL

SOURCE CODE

```cpp
//main.cpp
#include <iostream>
#include "HOA5P1_ILOB.h"

int main() {

   qNode<char> *front =nullptr;
   qNode<char> *back = nullptr;

   std::cout << "Current Queue: ";
   enqueue(&front, &back, 'J');
   enqueue(&front, &back, 'A');
   enqueue(&front, &back, 'V');
   enqueue(&front, &back, 'E');
   display(front);
   dequeue(&front, &back);
   std::cout << front->data << " " << back->data << std::endl;

   return 0;
}
```

```cpp
//main.h
#ifndef OBIAS_HOA5P1_ILOB_H
#define OBIAS_HOA5P1_ILOB_H

template <typename T>
class qNode {
public:
   T data;
   qNode* next;
};

//creating a new node
template <typename T>
qNode<T>* new_node(T newData) {
   qNode<T>* newNode = new qNode<T>;
   newNode->data = newData;
   newNode->next = nullptr;
   return newNode;
}

template <typename T>
void enqueue(qNode<T> **frontPtr, qNode<T> **backPtr, T newData) {
   //create a new node
   qNode<T> *newNode = new_node<T>(newData);

   //inserting to an empty queue
   if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
       (*frontPtr) = newNode;
       (*backPtr) = newNode;
   }

   //inserting an item into a non empty queue
   (*backPtr)->next = newNode; //point the backPtr next to the newNode
   (*backPtr) = newNode;
}

template <typename T>
void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr) {
   //create a temporary node to store the node to be deleted
   qNode<T> *deleteNode = nullptr;
   deleteNode = (*frontPtr);

   //check if the queue is only 1 node
   if ((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr) {
       (*frontPtr) = nullptr;
       (*backPtr) = nullptr;
       delete deleteNode;
       return;
   }

   //deleting of the node
   (*frontPtr) = deleteNode->next;
   delete deleteNode;
```

```
}

//display all elements in the list
template <typename T>
void display(qNode<T>* front) {
   //create a temporary node
   qNode<T> *displayNode = front;


   while (displayNode != nullptr) {
       std::cout << displayNode->data;
       if (displayNode->next != nullptr)
           std::cout << " -> ";

       displayNode = displayNode ->next;
   }

   std::cout << std::endl;
}

//return the front variable

#endif
```

OUTPUT

```
C:\Users\javej\OneDrive\Documents\OBIAS\cmake-build-debug\HOA5p1_ILOB.exe
Current Queue: J -> A -> V -> E
A E


Process finished with exit code 0
```

ANALYSIS

In this program, it uses a linked-list queue where the characters 'J', 'A', 'V', and 'E' are added using the enqueue() function, the display() function shows the elements in order using arrows, while dequeue() removes the first element 'J' from the queue. After removing it, the front becomes 'A' and the back stays 'E'.

Table 5.2 ILOB.1: Linked List Implementation

SOURCE CODE

```
//main.cpp
#include <iostream>
```

```cpp
#include "HOA5p1_ILOB_Arr.h"

int main() {
   //create and object queue
   queueArr<int> q(6);
   q.Enqueue(1);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << "
q_size: " << q.Size() << std::endl;
   q.Enqueue(2);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << "
q_size: " << q.Size() << std::endl;
   q.Enqueue(3);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << "
q_size: " << q.Size() << std::endl;
   q.Enqueue(4);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << "
q_size: " << q.Size() << std::endl;
   q.Enqueue(5);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << "
q_size: " << q.Size() << std::endl;
   q.Enqueue(6);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << "
q_size: " << q.Size() << std::endl;
   q.Enqueue(7);
   q.Dequeue();
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << "
q_size: " << q.Size() << std::endl;
   q.Dequeue();
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << "
q_size: " << q.Size() << std::endl;
   q.Dequeue();
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << "
q_size: " << q.Size() << std::endl;
   q.Clear();

   queueArr<int> q2(q);

   std::cout << "\nCopy Constructor" << std::endl;
    std::cout << "q2_front: " << q2.Front() << " q2_back: " << q2.Back() <<
" q2_size: " << q2.Size() << std::endl;
   queueArr<int> q3(6);

   q3 = q;
   std::cout << "\nCopy Assignment Operator" << std::endl;
    std::cout << "q3_front: " << q3.Front() << " q3_back: " << q3.Back() <<
" q3_size: " << q3.Size() << std::endl;

   return 0;
}
```

```cpp
//main.h
#ifndef OBIAS_HOA5P1_ILOB_Arr_H
```

```cpp
#define OBIAS_HOA5P1_ILOB_Arr_H

#include <iostream>

//Array-Based Circular Queue
template <typename T>
class queueArr {
   private:
       //create a pointer array:
       T *q_array;
       size_t q_capacity;
       size_t q_size;
       int q_front;
       int q_back;

   public:
       //prototypes:
       queueArr(size_t capacity);
       bool Empty();
       bool Full();

       size_t Size();
       void Clear();

       T Front();
       T Back();

       void Enqueue(T newData);
       void Dequeue();

       ~queueArr();

       //tasks:
       queueArr(const queueArr &other);
       queueArr &operator = (const queueArr &other);
};

//constructor:
template <typename T>
queueArr<T>::queueArr(size_t capacity) {

   //capacity of the array
   q_capacity = capacity;

   //current size of the array
   q_size = 0;

   //initial front and back index
   q_front = 0;
   q_back = -1;

   //newly allocated array
   q_array = new T[q_capacity];
```

```
    std::cout << "Constructor class was called\n";
}

//destructor:
template <typename T>
queueArr<T>::~queueArr() {
    delete [] q_array;
};

//empty:
template <typename T>
bool queueArr<T>::Empty() {
    return (q_size == 0);
}

//full:
template <typename T>
bool queueArr<T>::Full() {
    return (q_size == q_capacity);
}

//clear:
template <typename T>
void queueArr<T>::Clear() {
    q_front = 0;
    q_back = -1;
    q_size = 0;
}

//front:
template <typename T>
T queueArr<T>::Front() {
    //check and display if the queue is empty
    if(Empty()) {
        std::cout << "The queue is empty\n";
        return T();
    }

    //return the front item
    return q_array[q_front];
}

//back:
template <typename T>
T queueArr<T>::Back() {
    //check and display if the queue is empty
    if(Empty()) {
        std::cout << "The queue is empty\n";
        return T();
    }

    //return the back item
    return q_array[q_back];
```

```cpp
}

//size:
template <typename T>
size_t queueArr<T>::Size() {
    return q_size;
}

//enqueue:
template <typename T>
void queueArr<T>::Enqueue(T newData) {
    //check if the queue if full
    if(Full()) {
        std::cout << "Queue is full" << std::endl;
        return;
    }

    //compute for the q_back of the queue array
    q_back = (q_back + 1) % q_capacity;
    // 5 > 0, 6 > 1, 7 > 2, clue:use modulo

    //assign the new value to the back of the queue array
    q_array[q_back] = newData;

    //increment the q_size
    q_size++;
}

//dequeue:
template <typename T>
void queueArr<T>::Dequeue() {
    //check if its empty
    if(Empty()) {
        std::cout << "Queue is empty" << std::endl;
        return;
    }

    //compute for the q_front to move it circulary
    q_front = (q_front + 1) % q_capacity;

    //decrement the q_size
    q_size--;
}

//copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr &other) {
    //copy properties from the other
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
```

```
    //allocate new memory for the array
    q_array = new T[q_capacity];

    //copy the elements from the other queue
    for (size_t i = 0; i < q_size; i++) {
        size_t index = (q_front + q_back) % q_capacity;
        q_array[index] = other.q_array[index];
    }
}

//copy assignment operator
template <typename T>
queueArr<T> &queueArr<T>::operator=(const queueArr<T> &other) {
    if (this != &other) {
        //clean up existing memory
        delete [] q_array;

        //copy properties from the other
        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;

        //allocate new memory for the array
        q_array = new T[q_capacity];

        //copy the elements from the other queue
        for (size_t i = 0; i < q_size; i++) {
            size_t index = (q_front + q_back) % q_capacity;
            q_array[index] = other.q_array[index];
        }
    }
    return *this;
}

#endif
```

OUTPUT

```
C:\Users\javej\OneDrive\Documents\OBIAS\cmake-build-debug\HOA5p1_ILOB_Arr.exe
Constructor class was called
q_front: 1 q_back: 1 q_size: 1
q_front: 1 q_back: 2 q_size: 2
q_front: 1 q_back: 3 q_size: 3
q_front: 1 q_back: 4 q_size: 4
q_front: 1 q_back: 5 q_size: 5
q_front: 1 q_back: 6 q_size: 6
Queue is full
q_front: 2 q_back: 6 q_size: 5
q_front: 3 q_back: 6 q_size: 4
q_front: 4 q_back: 6 q_size: 3

--- Copy Constructor ---
q2_front: The queue is empty
0 q2_back: The queue is empty
0 q2_size: 0
Constructor class was called

--- Copy Assignment Operator ---
q3_front: The queue is empty
0 q3_back: The queue is empty
0 q3_size: 0

Process finished with exit code 0
```

ANALYSIS

In this program, it creates the array-based circular queue that has a capacity of 6 and it uses the Enqueue() to add numbers from 1 to 6 into the queue, the Front(), Back(), and Size() functions are used to check the first element, last element, and number of elements, while Dequeue() removes elements from the front. Also, it uses Clear(), the copy constructor, and copy assignment operator to manage and copy the queue.

Table 5.3 ILOB.2: Array-Based Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

SOURCE CODE

```
//main.cpp
#include <iostream>
```

```cpp
#include "HOA5p1_ILOC_printer.h"

int main() {
   //create a printer
   Printer printer;

   //add printing jobs
   printer.AddJob(Job(1, "Jave ", 5));
   printer.AddJob(Job(2, "Mike", 10));
   printer.AddJob(Job(3, "Yazi", 3));
   printer.AddJob(Job(4, "Dane", 7));

   //process all jobs
   printer.ProcessJobs();

   return 0;
}
```

```cpp
//job.h
#ifndef HOA5P1_ILOC_JOB_H
#define HOA5P1_ILOC_JOB_H

#include <iostream>
#include <string>

//Job class
class Job {
   private:
       //job information
       int jobID;
       std::string userName;
       int pages;

   public:
       //prototypes
       Job();
       Job(int id, std::string name, int numPages);
       void Display();
};

//constructor
Job::Job() {
   //initial values
   jobID = 0;
   userName = "";
   pages = 0;
}

//constructor with values
Job::Job(int id, std::string name, int numPages) {
   //assign values
   jobID = id;
   userName = name;
```

```
    pages = numPages;
}

//display job information
void Job::Display() {
    std::cout << "Job ID: " << jobID
              << ", User: " << userName
              << ", Pages: " << pages << std::endl;
}

#endif
```

```
//printer.h
#ifndef HOA5P1_ILOC_PRINTER_H
#define HOA5P1_ILOC_PRINTER_H

#include <iostream>
#include "HOA5p1_ILOC_job.h"

//array circular queue
template <class T>
class queueArr {
private:
    //create a pointer array
    T *q_array;
    size_t q_capacity;
    size_t q_size;
    int q_front;
    int q_back;

public:
    //prototypes
    queueArr(size_t capacity);
    bool Empty();
    bool Full();
    size_t Size();
    void Clear();
    T Front();
    T Back();
    void Enqueue(T newData);
    void Dequeue();
    ~queueArr();
};

//constructor
template <class T>
queueArr<T>::queueArr(size_t capacity) {
    //capacity of the array
    q_capacity = capacity;
    //current size of the queue
    q_size = 0;
    //initial front and back index
    q_front = 0;
```

```
    q_back = -1;
    //newly allocated array
    q_array = new T[q_capacity];
}

//destructor
template <class T>
queueArr<T>::~queueArr() {
    delete [] q_array;
}

//empty
template <class T>
bool queueArr<T>::Empty() {
    //check if the queue is empty
    return (q_size == 0);
}

//full
template <class T>
bool queueArr<T>::Full() {
    //check if the queue is full
    return (q_size == q_capacity);
}

//size
template <class T>
size_t queueArr<T>::Size() {
    //return the queue size
    return q_size;
}

//clear
template <class T>
void queueArr<T>::Clear() {
    //reset front and back
    q_front = 0;
    q_back = -1;
    q_size = 0;
}

//front
template <class T>
T queueArr<T>::Front() {
    //check if the queue is empty
    if(Empty()) {
        std::cout << "The queue is empty\n";
        return T();
    }
    //return the front item
    return q_array[q_front];
}
```

```cpp
//back
template <class T>
T queueArr<T>::Back() {
    //check if the queue is empty
    if(Empty()) {
        std::cout << "The queue is empty\n";
        return T();
    }
    //return the back item
    return q_array[q_back];
}

//enqueue
template <class T>
void queueArr<T>::Enqueue(T newData) {
    //check if the queue is full
    if(Full()) {
        std::cout << "Queue is full" << std::endl;
        return;
    }
    //move the back circularly
    q_back = (q_back + 1) % q_capacity;
    //assign the new value to the back
    q_array[q_back] = newData;
    //increment the queue size
    q_size++;
}

//dequeue
template <class T>
void queueArr<T>::Dequeue() {
    //check if the queue is empty
    if(Empty()) {
        std::cout << "Queue is empty" << std::endl;
        return;
    }
    //move the front circularly
    q_front = (q_front + 1) % q_capacity;
    //decrement the queue size
    q_size--;
}

//printer class
class Printer {
private:
    //create a queue for the jobs
    queueArr<Job> jobs;

public:
    //prototypes
    Printer();
    void AddJob(Job newJob);
    void ProcessJobs();
```

```
};

//constructor
Printer::Printer() : jobs(10) {
}

//add a new job
void Printer::AddJob(Job newJob) {
   //add the job to the queue
   jobs.Enqueue(newJob);
   std::cout << "Job added: ";
   newJob.Display();
}

//process all jobs
void Printer::ProcessJobs() {
   std::cout << "\nProcessing jobs...\n" << std::endl;
   //check if there are still jobs
   while(!jobs.Empty()) {
       //get the first job
       std::cout << "Printing: ";
       jobs.Front().Display();
       //remove the first job
       jobs.Dequeue();
       std::cout << "Job completed!\n" << std::endl;
   }
}

#endif
```

OUTPUT

```
C:\Users\javej\OneDrive\Documents\OBIAS\cmake-build-debug\HOA5p1_ILOC.exe
Job added: Job ID: 1, User: Jave , Pages: 5
Job added: Job ID: 2, User: Mike, Pages: 10
Job added: Job ID: 3, User: Yazi, Pages: 3
Job added: Job ID: 4, User: Dane, Pages: 7

Processing jobs...

Printing: Job ID: 1, User: Jave , Pages: 5
Job completed!

Printing: Job ID: 2, User: Mike, Pages: 10
Job completed!

Printing: Job ID: 3, User: Yazi, Pages: 3
Job completed!

Printing: Job ID: 4, User: Dane, Pages: 7
Job completed!


Process finished with exit code 0
```

CONCLUSION

I learned in this activity on how a circular queue can be used to manage printing jobs and how Enqueue() and Dequeue() work when adding and processing jobs. Also in the procedure, it helps me to understand how the queue connects with the Printer and Job classes to process each job in the correct order. I think I did well in completing the activity, but I still need to improve my understanding of circular queues and how the different functions work together.

Table 5.4 ILO C: Solve problems using queue implementation

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.